UNIVERSIDAD
DE PINAR DEL RÍO
HERMANOS SAÍZ MONTES DE OCA

FACULTAD DE CIENCIAS TÉCNICAS · DEPARTAMENTO DE INFORMÁTICA

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

GESTIÓN ACADÉMICA Y CARGA DOCENTE

MULTILISTA, COLA, PILA Y RECURSIVIDAD EN JAVA

Estructuras de Datos · 2do Examen Parcial

EQUIPO 1

YAISEL PINOS DURÁN · MERLYN SOSA BORGES · GEONEL SÁNCHEZ
JORGE LUIS REYES PÉREZ · RAY SANJUDO MESA

CURSO 2026-2027

2026

Senda para este tiempo

UNIVERSIDAD DE PINAR DEL RÍO
HERMANOS SAÍZ MONTES DE OCA

CONTENIDO

1 Problema y objetivo 2
2 Arquitectura 3
3 Multilista 4
4 Método recursivo: destacados 6
5 Cola de solicitudes (FIFO) 7
6 Pila de historial (LIFO) 9
7 Sistema integrado 11
8 Excepciones 14
9 Reglas de negocio 15
10 Pruebas 17
11 Compilar y ejecutar 18
12 Declaración de uso de IA 19
A Anexo: las 164 pruebas 20

1 PROBLEMA Y OBJETIVO

El departamento necesita controlar la carga docente de sus profesores: quiénes son, qué categoría y grado científico tienen, cómo fueron evaluados y cuántas horas imparten en cada asignatura. Además, los datos de los profesores cambian a menudo, por pedido de los propios profesores o por correcciones, y esos cambios tienen que poder auditarse y revertirse.

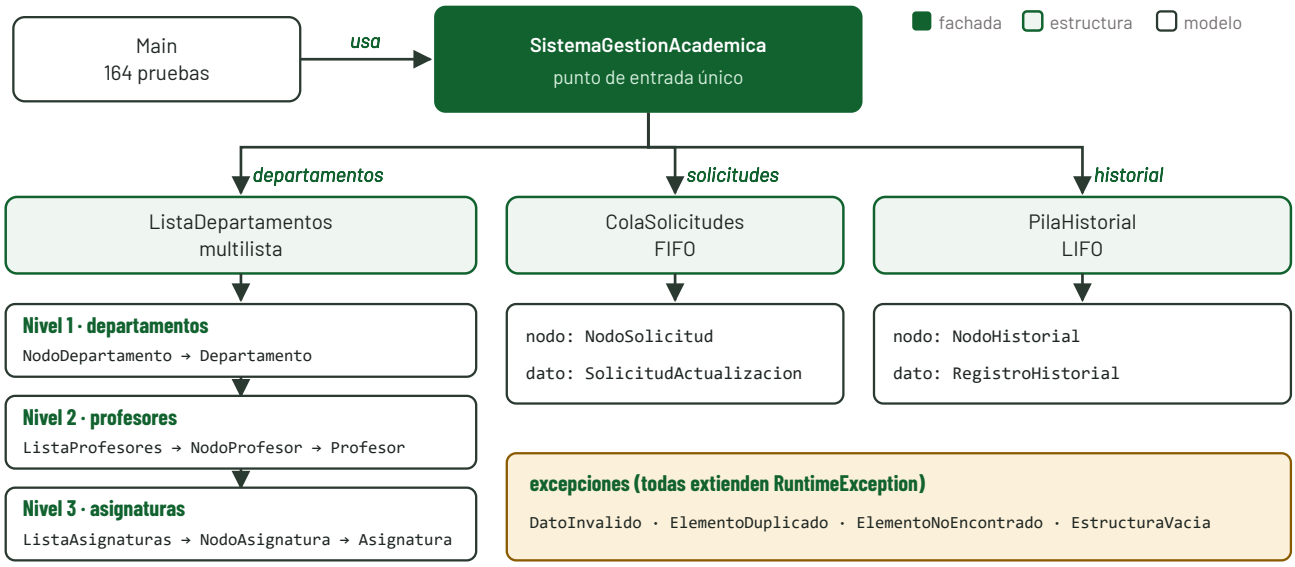
Este proyecto, del **Equipo 1**, resuelve ese problema en Java con estructuras de datos dinámicas hechas a mano (nodos y referencias, sin colecciones de `java.util`) y con métodos recursivos en lugar de bucles. La tabla resume qué pide la orden del 2do Examen Parcial y dónde se cumple cada punto:

LO QUE PIDE LA ORDEN	CÓMO LO RESUELVE EL PROYECTO
Multilista Departamento → Profesores → Asignaturas. Del profesor: nombre, categoría docente, grado científico, edad, evaluación y sus asignaturas (nombre y horas).	`ListaDepartamentos` → `ListaProfesores` → `ListaAsignaturas`, cada una con su propio tipo de nodo. Sección 3.
Pila (LIFO) con el historial de revisiones o modificaciones de carga horaria, para auditar o deshacer el último cambio.	`PilaHistorial` guarda cada cambio; `deshacerUltimoCambio()` lo revierte. Sección 6.
Cola (FIFO) de atención de solicitudes de actualización de datos de los profesores.	`ColaSolicitudes` y `procesarSiguienteSolicitud()`, con estados pendiente, procesada y rechazada. Sección 5.
Método recursivo que recorra la multilista y cuente los profesores destacados (más de 120 horas en total).	`contarProfesoresDestacados()` recursivo en los tres niveles, sin `for` ni `while`. Sección 4.
Pruebas positivas y negativas (listas vacías, elementos no encontrados, excepciones).	164 pruebas en `Main.java`: 164 exitosas, 0 fallidas. Sección 10 y anexo.

Senda para este tiempo

2 ARQUITECTURA

El proyecto tiene 21 clases en cuatro paquetes, unas 2.000 líneas sin contar las pruebas. `SistemaGestionAcademica` es la fachada: `Main` y cualquier otro cliente solo hablan con ella, y ella coordina las tres estructuras.



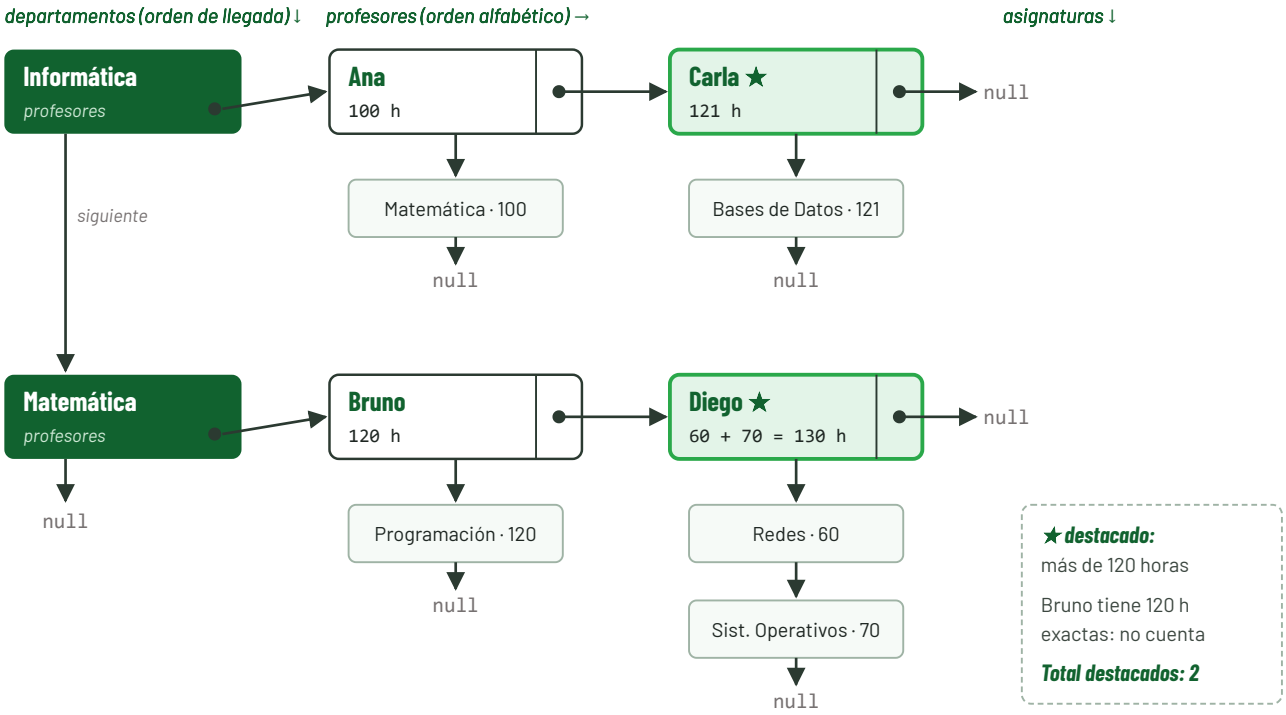
PAQUETE	CLASES	RESPONSABILIDAD
`gestionacademica`	`Main`, `SistemaGestionAcademica`	Pruebas y fachada del sistema.
`modelo`	`Departamento`, `Profesor`, `Asignatura`, `SolicitudActualizacion`, `RegistroHistorial`	Los datos. Cada clase valida sus atributos y lanza DatoInvalidoException si algo no es válido.
`estructuras`	5 nodos, 3 listas, `ColaSolicitudes`, `PilaHistorial`	Las estructuras enlazadas y sus recorridos recursivos.
`excepciones`	4 excepciones propias	Distinguir dato inválido, duplicado, no encontrado y estructura vacía.

DECISIONES QUE SE APLICAN EN TODO EL PROYECTO

- **Nodos enlazados hechos a mano.** Ninguna estructura usa arreglos ni colecciones de Java.
- **Recursividad en lugar de bucles.** En los paquetes `modelo` y `estructuras` y en `SistemaGestionAcademica` no hay ni un `for` ni un `while`: todos los recorridos son recursivos.
- **Validar al crear.** Los objetos del modelo se validan en el constructor, así que nunca existe un profesor con evaluación 150 o una asignatura con 0 horas.
- **Excepciones con significado.** Cada tipo de error tiene su propia excepción, y las pruebas comprueban que salga exactamente la esperada.

3 MULTILISTA

La multilista tiene tres niveles. La lista de departamentos es la principal; cada departamento tiene su propia lista de profesores y cada profesor su propia lista de asignaturas. Así queda con los datos de `pruebasDepartamentos()`:



3.1 LAS TRES LISTAS

	ListaDepartamentos	ListaProfesores	ListaAsignaturas
Nodo	`NodoDepartamento`	`NodoProfesor`	`NodoAsignatura`
Inserción	`insertarFinal`: orden de llegada	`insertarOrdenado`: alfabético	`insertarOrdenado`: alfabético
Buscar / existe	`buscarRec`	`buscarRec`	`buscarRec`
Eliminar	`eliminarRec`	`eliminarRec`	`eliminarRec`
Listar	`listarRec`	`listarRec`	`listarRec`
Propio	`contarProfesoresDestacadosRec`	`contarProfesoresDestacadosRec`	`sumarHorasRec`, `actualizarHoras`

Las búsquedas no distinguen mayúsculas (`equalsIgnoreCase`) y el orden alfabético usa `compareToIgnoreCase`. Todas las operaciones son O(n). Insertar un nombre que ya está lanza `ElementoDuplicadoException`; buscar o eliminar uno que no está lanza `ElementoNoEncontradoException`.

3.2 EL PATRÓN RECURSIVO DE LAS LISTAS

Cada operación tiene un método público que valida y un método privado `…Rec(nodo, …)` que resuelve el nodo actual y le pasa el resto de la lista a la siguiente llamada. El caso base siempre es `nodo == null`. Insertar y eliminar devuelven el nuevo inicio de la sublista, y cada nodo se reengancha con `nodo.setSiguiente(…Rec(nodo.getSiguiente(), …))`. Así no hace falta guardar una referencia al nodo anterior, y el mismo código sirve para insertar al principio, en el medio o al final.

3.3 VALIDACIONES DEL MODELO

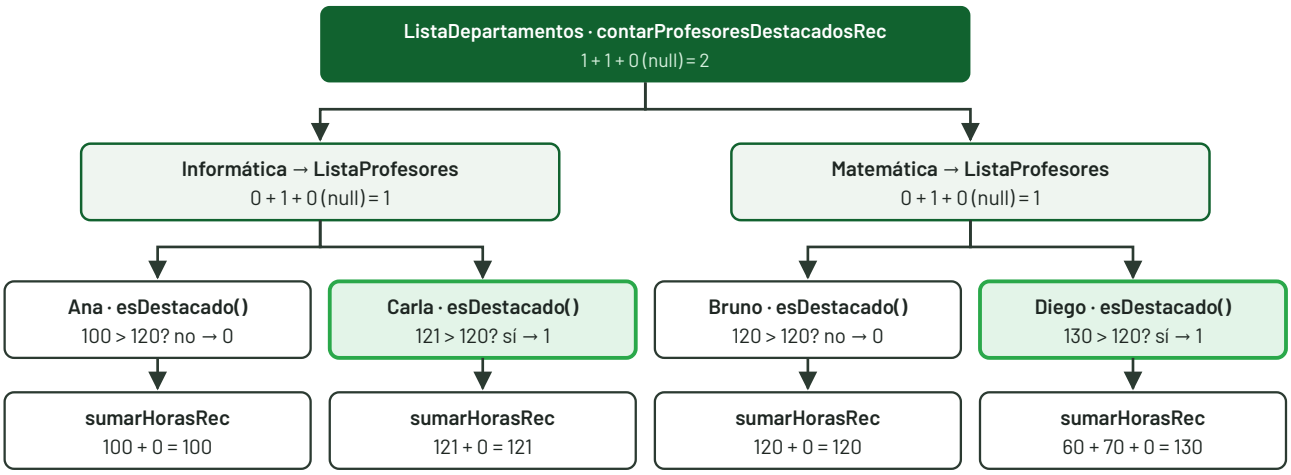
CLASE	REGLA	SI NO SE CUMPLE
`Departamento`	nombre no vacío	DatoInvalidoException
`Profesor`	nombre, categoría docente y grado científico no vacíos; edad ≥ 0; evaluación entre 0 y 100	DatoInvalidoException
`Asignatura`	nombre no vacío; horas > 0	DatoInvalidoException
Las tres listas	no se admite un nodo con dato nulo ni un nombre repetido	DatoInvalidoException / ElementoDuplicadoException

4 MÉTODO RECURSIVO: PROFESORES DESTACADOS

La orden pide un algoritmo recursivo que recorra la multilista y cuente los profesores destacados, es decir, los que suman **más de 120 horas** entre todas sus asignaturas. El conteo baja por los tres niveles y cada nivel es recursivo por su cuenta:

1. `ListaDepartamentos` suma los destacados de cada departamento y sigue con el siguiente.
2. `ListaProfesores` suma 1 por cada profesor con `esDestacado()` verdadero.
3. `Profesor.esDestacado()` compara `getTotalHoras() > 120`, y el total sale de `ListaAsignaturas.sumarHorasRec`.

ÁRBOL DE LLAMADAS CON LOS DATOS DE LA SECCIÓN 3



Caso base en los tres niveles: si el nodo es null, la llamada devuelve 0.

> **EL LÍMITE ES ESTRICTO**
>
> Un profesor con 120 horas exactas no es destacado (Bruno en el ejemplo). Las pruebas lo cubren: en `pruebasDepartamentos()`, Matemática da 1 destacado porque Bruno tiene 120 h y Diego 130 h.

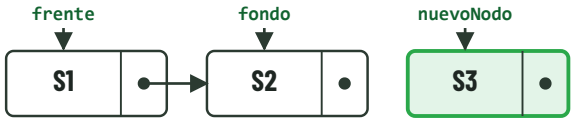
5 COLA DE SOLICITUDES (FIFO)

Los profesores piden cambios en sus datos (categoría, grado, edad, evaluación u horas de una asignatura). Cada pedido es una `SolicitudActualizacion` que espera su turno en `ColaSolicitudes`: se atiende primero la que llegó primero. La cola guarda dos referencias, `frente` (la próxima en salir) y `fondo` (la última que entró), así que encolar y desencolar cuestan O(1).

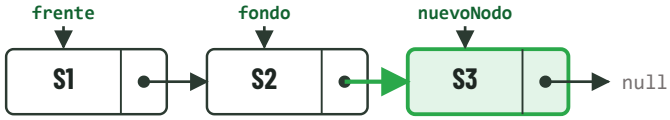
MÉTODO	QUÉ HACE	COSTO	EXCEPCIÓN
`enqueue(solicitud)`	Engancha un nodo nuevo detrás del fondo. Si la cola estaba vacía, frente y fondo apuntan al mismo nodo.	`O(1)`	DatoInvalidoException si es null
`dequeue()`	Saca la solicitud del frente. Si la cola queda vacía, también pone fondo = null.	`O(1)`	EstructuraVaciaException si está vacía
`peek()`	Devuelve la del frente sin sacarla.	`O(1)`	EstructuraVaciaException si está vacía
`estaVacia()` · `getTotal()`	frente == null · contador.	`O(1)`	—
`listar()`	Recorre del frente al fondo con listarRec.	`O(n)`	—

5.1 enqueue, PASO A PASO

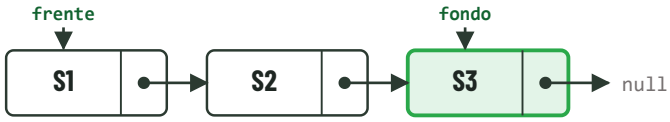
```
NodoSolicitud nuevoNodo =
    new NodoSolicitud(solicitud);
```




```
fondo.setSiguiente(nuevoNodo);
```

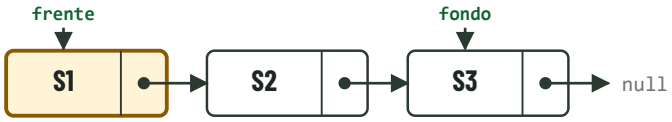



```
fondo = nuevoNodo;
total++;      // total = 3
```

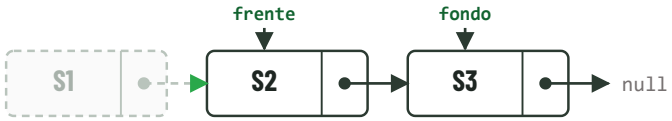


5.2 dequeue, *PASO A PASO*

```
SolicitudActualizacion solicitud =
    frente.getDato();
```




```
frente = frente.getSiguiente();
total--;      // total = 2
```

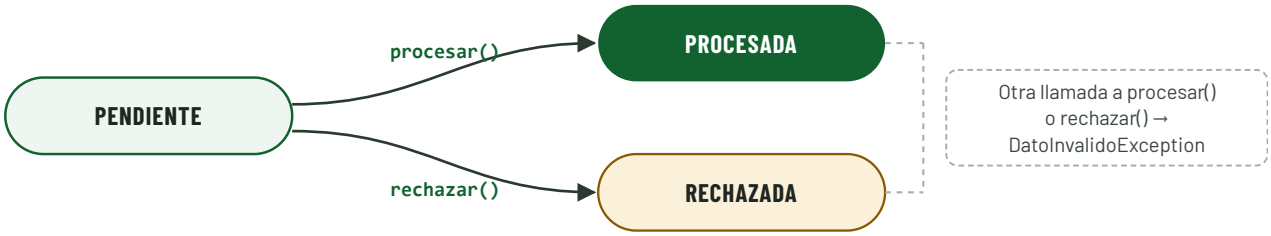



```
if (frente == null) fondo = null;
return solicitud;
```



El último paso es el detalle que más se olvida: si sale la única solicitud, `frente` queda en `null`, y si `fondo` siguiera apuntando al nodo viejo, el próximo `enqueue` lo engancharía a un nodo que ya no está en la cola.

5.3 ESTADOS Y VALIDACIONES DE UNA SOLICITUD



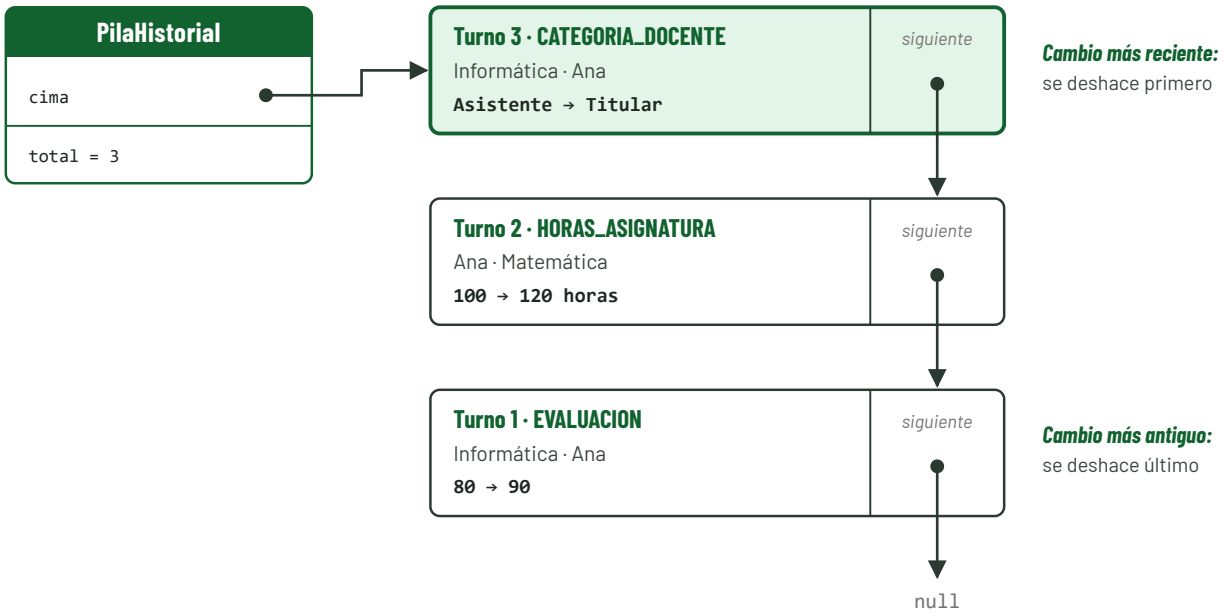
REGLA AL CREAR LA SOLICITUD	EJEMPLO RECHAZADO
El id debe ser mayor que 0; departamento y profesor no vacíos.	id = 0
El tipo debe ser CATEGORIA_DOCENTE, GRADO_CIENTIFICO, EDAD, EVALUACION u HORAS_ASIGNATURA.	tipo «SALARIO»
Las solicitudes de horas deben indicar la asignatura; las demás no pueden indicarla.	horas sin asignatura
El nuevo valor se valida según el tipo: edad entero ≥ 0, evaluación entre 0 y 100, horas enteras > 0.	edad «abc», horas −5

Todas lanzan `DatoInvalidoException`, antes de que la solicitud llegue a la cola.

6 PILA DE HISTORIAL (LIFO)

Cada cambio que se hace sobre un profesor o una asignatura se guarda como un `RegistroHistorial` en la cima de `PilaHistorial`: turno, descripción, departamento, profesor, asignatura (solo en cambios de horas), tipo de modificación, valor anterior y valor nuevo. El último cambio que entra es el primero que se deshace, igual que `Ctrl+Z` en un editor.

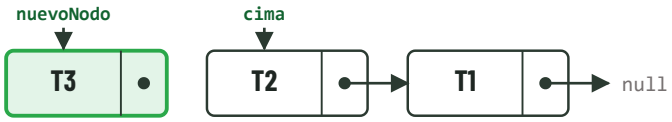
LA PILA EN MEMORIA, CON LOS REGISTROS QUE USA PRUEBASPILA()



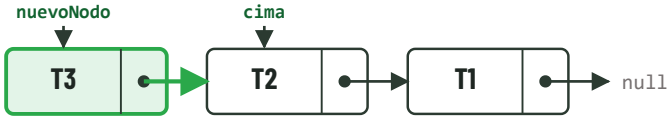
MÉTODO	QUÉ HACE	COSTO	EXCEPCIÓN
`push(registro)`	Crea un nodo, lo pone encima de la cima y lo convierte en la nueva cima.	`O(1)`	DatoInvalidoException si es null
`pop()`	Devuelve el registro de la cima y baja la cima al siguiente nodo.	`O(1)`	EstructuraVaciaException si está vacía
`peek()`	Devuelve el registro de la cima sin quitarlo.	`O(1)`	EstructuraVaciaException si está vacía
`estaVacia()` · `getTotal()`	cima == null · contador.	`O(1)`	—
`listar()`	Recorre de la cima al fondo con listarRec: sale primero el cambio más reciente.	`O(n)`	—

6.1 push, *PASO A PASO*

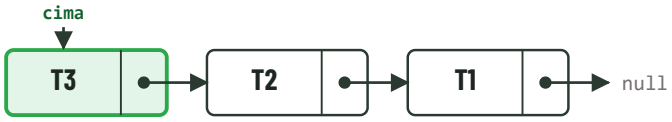
```
NodoHistorial nuevoNodo =
    new NodoHistorial(registro);
```




```
nuevoNodo.setSiguiente(cima);
```

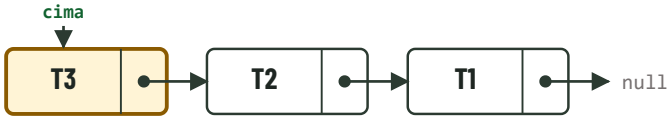



```
cima = nuevoNodo;
total++;      // total = 3
```

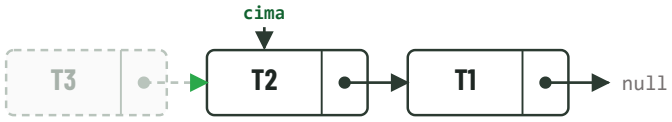


6.2 pop, *PASO A PASO*

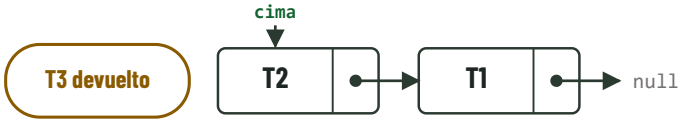
```
RegistroHistorial registro =
    cima.getDato();
```




```
cima = cima.getSiguiente();
total--;      // total = 2
```




```
return registro;
```



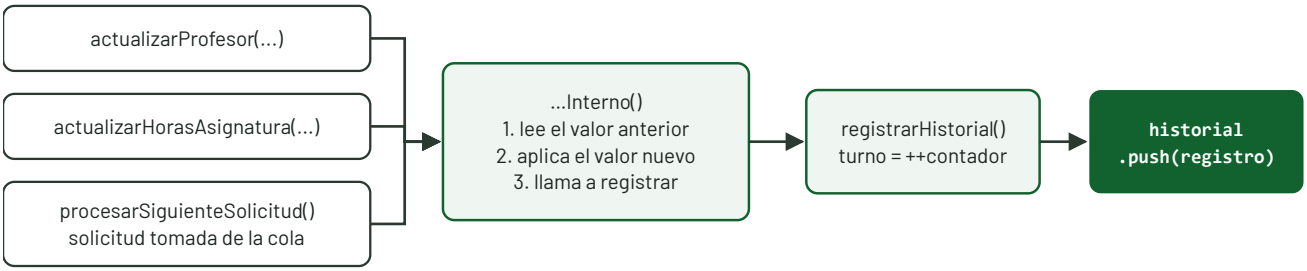
`RegistroHistorial` no tiene setters públicos: lo que entra al historial no cambia, y para deshacer se usa `valorAnterior` tal como se guardó. Su constructor valida que el turno sea mayor que 0 y que ningún texto obligatorio venga vacío.

7 SISTEMA INTEGRADO

`SistemaGestionAcademica` une las tres estructuras. Estos son sus métodos públicos, agrupados por la estructura que usan:

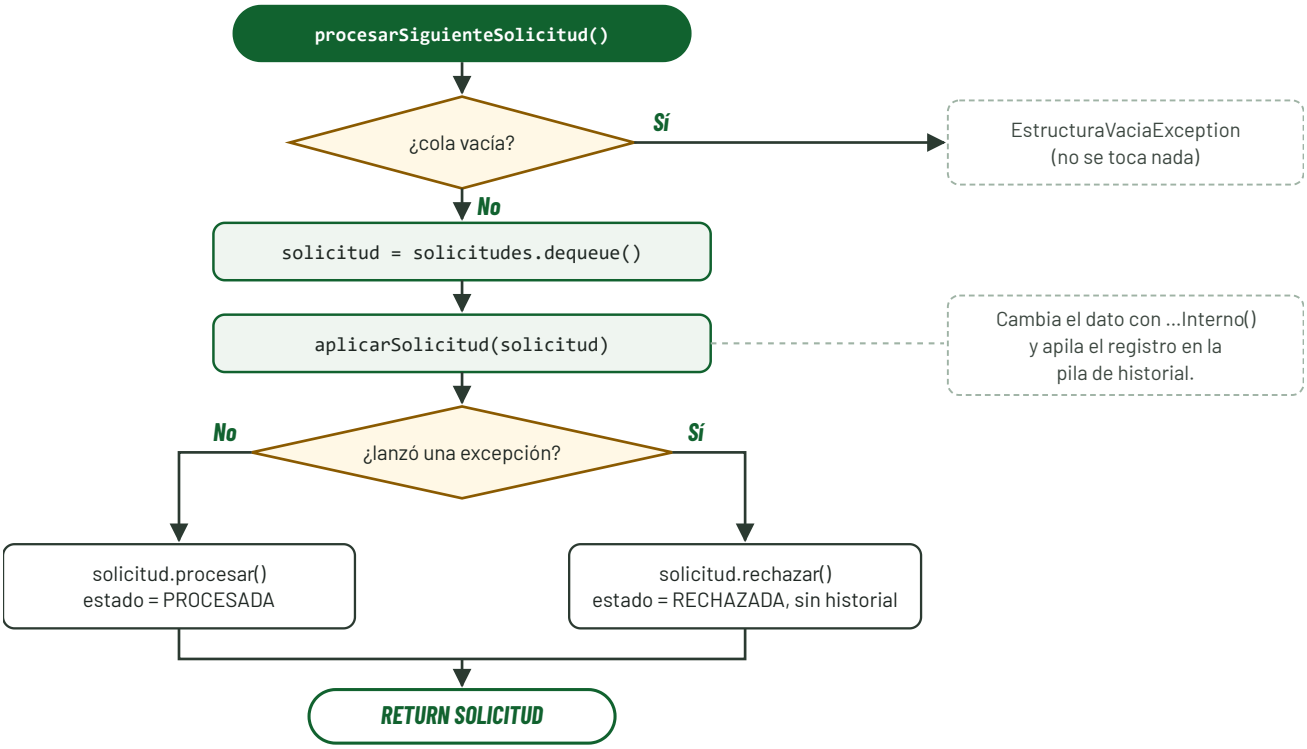
GRUPO	MÉTODOS	ESTRUCTURA
Datos	`crearDepartamento` `crearProfesor` `crearAsignatura` `buscar…` `eliminar…` `listar…`	Multilista
Cambios directos	`actualizarProfesor` `actualizarHorasAsignatura`	Multilista + pila
Solicitudes	`registrarSolicitud` `procesarSiguienteSolicitud` `listarSolicitudes` `getTotalSolicitudes`	Cola (+ pila al procesar)
Historial	`verUltimoCambio` `deshacerUltimoCambio` `listarHistorial` `getTotalHistorial`	Pila
Conteo	`contarProfesoresDestacados`	Recursivo sobre la multilista

7.1 CÓMO ENTRA UN CAMBIO A LA PILA

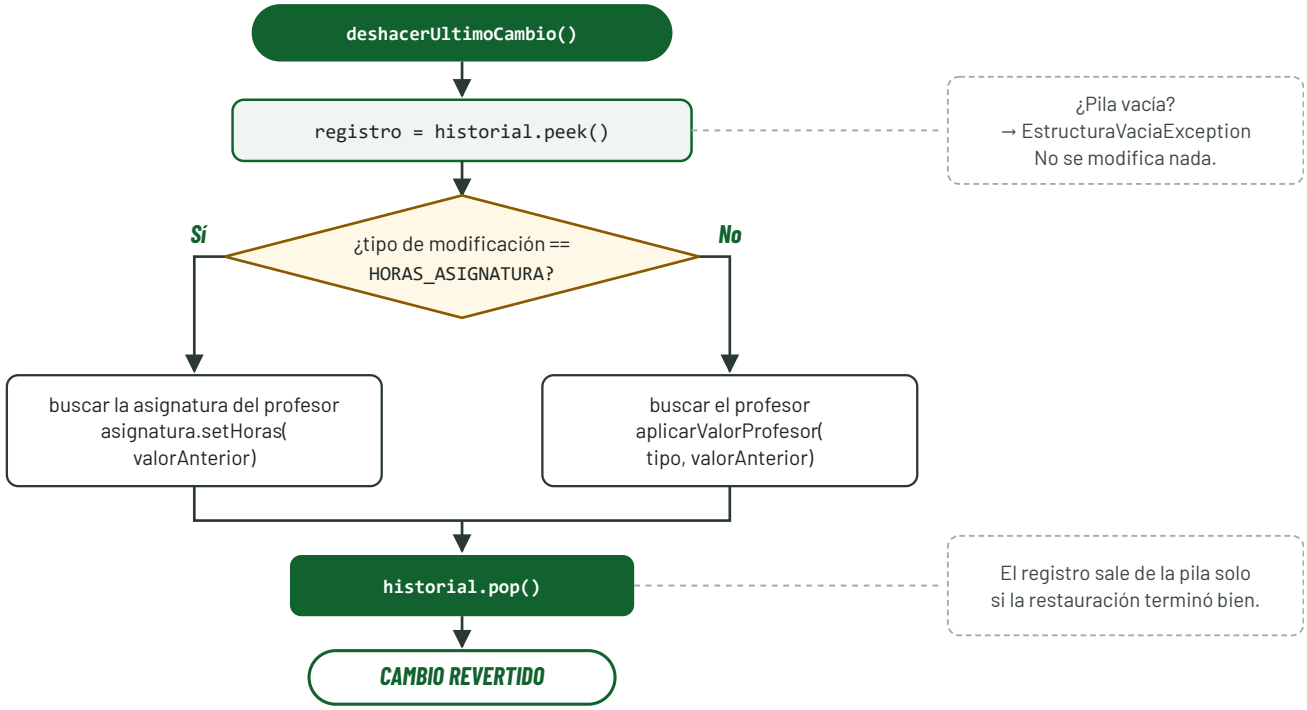


Los cambios directos y los que vienen de la cola pasan por el mismo camino: se lee el valor anterior, se aplica el nuevo y se apila el registro. El turno lo da un contador del sistema que nunca retrocede.

7.2 procesarSiguienteSolicitud()



7.3 deshacerUltimoCambio()



> **¿POR QUÉ PRIMERO `peek()` Y AL FINAL `pop()` ?**
>
> Si la restauración falla (por ejemplo, porque el profesor se eliminó después del cambio), la excepción sale antes de llegar a `pop()` y el registro sigue en la pila. Deshacer tampoco genera un registro nuevo: restaura con `setHoras` o `aplicarValorProfesor` directamente, sin pasar por `registrarHistorial()` .

7.4 LAS TRES ESTRUCTURAS JUNTAS `pruebasSistema()`

Profesora Ana, de Informática: evaluación 80 y Matemática con 100 horas al empezar.

#	OPERACIÓN	COLA	PILA (CIMA PRIMERO)	EVAL.	HORAS	DEST.
1	actualizarHorasAsignatura → 121	`vacía`	`T1 HORAS`	80	121	1
2	deshacerUltimoCambio()	`vacía`	`vacía`	80	100	0
3	actualizarProfesor EVALUACION → 90	`vacía`	`T2 EVALUACION`	90	100	0
4	deshacerUltimoCambio()	`vacía`	`vacía`	80	100	0
5	registrarSolicitud EVALUACION 95	`S1`	`vacía`	80	100	0
6	procesarSiguienteSolicitud()	`vacía`	`T3 EVALUACION`	95	100	0
7	registrarSolicitud HORAS 130	`S2`	`T3`	95	100	0
8	procesarSiguienteSolicitud()	`vacía`	`T4 HORAS · T3`	95	130	1
9	deshacerUltimoCambio()	`vacía`	`T3 EVALUACION`	95	100	0
10	deshacerUltimoCambio()	`vacía`	`vacía`	80	100	0
11	registrarSolicitud con profesor «NoExiste»	`S3`	`vacía`	80	100	0
12	procesarSiguienteSolicitud() → RECHAZADA	`vacía`	`vacía`	80	100	0

8 EXCEPCIONES

Las cuatro excepciones del paquete `excepciones` extienden `RuntimeException`. Cada una indica un tipo de problema distinto, y así quien llama sabe qué pasó sin leer el mensaje.

EXCEPCIÓN	CUÁNDO SE LANZA	EJEMPLOS DEL PROYECTO
`DatoInvalidoException`	Un dato es nulo, vacío o está fuera de rango.	horas ≤ 0; evaluación fuera de 0–100; edad negativa; tipo de solicitud inválido; push(null); procesar una solicitud ya atendida
`ElementoDuplicadoException`	Se inserta un nombre que ya existe en esa lista.	departamento, profesor o asignatura repetidos (sin distinguir mayúsculas)
`ElementoNoEncontradoException`	Se busca, modifica o elimina algo que no está.	buscar o eliminar un profesor inexistente; actualizar horas de una asignatura que no existe
`EstructuraVaciaException`	Se saca o consulta algo de una estructura vacía.	pop() o peek() en la pila vacía; dequeue() o peek() en la cola vacía; procesar con la cola vacía

Una excepción no se captura dentro del sistema: `procesarSiguienteSolicitud()`. Ahí cualquier error al aplicar la solicitud se convierte en una solicitud **RECHAZADA**, para que una solicitud mala no bloquee la cola.

9 REGLAS DE NEGOCIO

La mayoría de las reglas de negocio del equipo (RN), los requerimientos funcionales (RF) y los no funcionales (RNF) ya se explican en las secciones anteriores. Esta tabla indica dónde está cada grupo:

REGLAS	TEMA	SECCIÓN
RN-01 a RN-04 · RN-07 · RN-09 a RN-22	Departamentos, profesores, asignaturas, orden y validaciones	3 y 4
RN-23 a RN-38	Solicitudes: validaciones, estados y orden FIFO	5 y 7
RN-39 a RN-51	Historial: registro de cambios, orden LIFO y deshacer	6 y 7
RN-53 a RN-57 · RF-49 a RF-52	Excepciones propias	8
RF-01 a RF-28	Gestión de la multilista y carga docente	3, 4 y 7
RF-29 a RF-48	Solicitudes e historial	5, 6 y 7
RNF-02 a RNF-19 · RNF-24	Robustez, integridad, consistencia, trazabilidad y complejidad	2, 3, 5, 6, 7 y 8

9.1 REGLAS QUE SE PRECISAN AQUÍ

ID	REGLA	DÓNDE SE CUMPLE
RN-05	Eliminar un departamento elimina también sus profesores y las asignaturas de esos profesores, porque están contenidos dentro de él.	`Departamento` contiene su `ListaProfesores`; cada `Profesor`, su `ListaAsignaturas`.
RN-06	Un profesor solo se registra dentro de un departamento que ya existe.	`crearProfesor` busca primero el departamento; si no está, lanza ElementoNoEncontradoException.
RN-08 · RN-20	No se repite el nombre de un profesor dentro del mismo departamento, ni el de una asignatura dentro del mismo profesor. En departamentos distintos sí puede haber dos profesores con el mismo nombre.	Cada departamento tiene su propia lista de profesores y cada profesor la suya de asignaturas.
RN-24 · RF-29	Las solicitudes reciben identificadores consecutivos: 1, 2, 3... Si una solicitud no es válida, no consume número.	`registrarSolicitud` usa el contador + 1 y solo lo incrementa si la solicitud se creó bien.
RN-28	Una solicitud de horas debe indicar el nombre de la asignatura. Que esa asignatura exista se comprueba al procesarla: si no existe, la solicitud queda RECHAZADA.	`SolicitudActualizacion` al crearla y `procesarSiguienteSolicitud` al atenderla.
RN-30	El nuevo valor de una solicitud no puede estar vacío.	`SolicitudActualizacion`, al crearla.
RN-45	En el historial, la asignatura es opcional (no se indica en cambios que no son de horas), pero si se indica no puede estar vacía.	`RegistroHistorial`, al crearlo.
RN-52	Eliminar departamentos, profesores o asignaturas no deja registro en el historial: solo las actualizaciones se pueden deshacer.	Los métodos eliminar... del sistema no pasan por registrarHistorial().
RN-58 · RNF-03	Los nombres se limpian de espacios al inicio y al final antes de guardarlos o buscarlos, y un nombre formado solo por espacios se rechaza.	Validación de nombres en las tres listas y en los constructores del modelo.

ID	REGLA	DÓNDE SE CUMPLE
RF-05	Se puede consultar cuántos departamentos hay registrados.	`getTotalDepartamentos`
RF-34	La primera solicitud se puede consultar sin sacarla de la cola.	`ColaSolicitudes.peek` . El sistema no tiene un método propio para esto.
RF-39	La cantidad de solicitudes que informa el sistema es la de las que esperan en la cola. Al procesarlas, baja hasta 0.	`getTotalSolicitudes` devuelve el total de la cola.
RNF-01 · RNF-20	Una operación inválida no cambia el estado: los datos se validan antes de modificar nada, y al deshacer se consulta la cima antes de quitarla.	Constructores del modelo, validación de valores antes de aplicarlos y peek() antes de pop().
RNF-06 · RNF-22	Los mensajes de error dicen qué dato o elemento causó el problema. Por ejemplo: «La asignatura "Redes" no fue encontrada.» o «La evaluación del profesor debe estar entre 0 y 100.»	Las cuatro excepciones propias.
RNF-21	Los listados muestran un elemento por línea con sus datos y, si la estructura está vacía, un mensaje claro como «Cola de solicitudes vacía.».	Los métodos listar... de cada estructura.
RNF-23	Todo funciona en memoria, sin base de datos ni archivos externos.	Todas las estructuras son nodos enlazados en memoria.
RNF-25	El diseño es adecuado para volúmenes académicos pequeños o medianos: las búsquedas recorren la lista (O(n)).	Listas enlazadas recorridas de forma recursiva.

10 PRUEBAS

Las pruebas están en `Main.java`, una función por módulo. Cada caso se ejecuta con el método auxiliar `probar(descripción, excepciónEsperada, acción)`: cuenta como exitoso si la acción lanza exactamente la excepción esperada, o ninguna cuando no se espera. Al final se imprime el total de exitosas y fallidas.

164	61	164 / 164	0
pruebas en total	esperan una excepción (casos negativos)	exitosas	fallidas

MÓDULO	MÉTODO EN MAIN.JAVA	PRUEBAS	POSITIVAS	NEGATIVAS	BORDE	RESULTADO
Asignaturas	`pruebasAsignaturas()`	**14**	5	9	0	✓ 14/14
Profesores	`pruebasProfesores()`	**28**	12	14	2	✓ 28/28
Departamentos	`pruebasDepartamentos()`	**23**	15	7	1	✓ 23/23
Pila de historial	`pruebasPila()`	**24**	10	10	4	✓ 24/24
Cola de solicitudes	`pruebasCola()`	**32**	12	15	5	✓ 32/32
Sistema integrado	`pruebasSistema()`	**43**	33	6	4	✓ 43/43
Total		**164**	**87**	**61**	**16**	✓ 164/164

Positivas: flujo normal. **Negativas**: esperan una excepción. **Borde**: estructuras vacías al empezar o después de vaciarlas. La lista completa está en el anexo.

10.1 QUÉ CUBREN

- **Flujos normales:** insertar, buscar, actualizar y eliminar en los tres niveles; orden alfabético y de llegada; orden FIFO de la cola y LIFO de la pila.
- **Datos inválidos:** nombres vacíos, nulos, horas en cero o negativas, evaluación mayor que 100, edad negativa, solicitudes con tipo o valor incorrecto.
- **Elementos que no existen o se repiten:** buscar, actualizar y eliminar inexistentes; insertar duplicados.
- **Estructuras vacías:** contar destacados sin datos, `pop` / `peek` / `dequeue` en vacío, procesar con la cola vacía, listar vacío.
- **Integración:** el conteo de destacados cambia al actualizar horas, al deshacer y al eliminar; una solicitud rechazada no deja historial.

CASOS SIN PRUEBA PROPIA

Llamar a `deshacerUltimoCambio()` con el historial vacío (por diseño, `peek()` lanza `EstructuraVaciaException`) y deshacer un cambio de un profesor que ya fue eliminado (la búsqueda lanza `ElementoNoEncontradoException` y el registro se queda en la pila).

11 COMPILAR Y EJECUTAR

Desde la raíz del repositorio, con un JDK 8 o superior (las pruebas usan lambdas):

```
$  javac -encoding UTF-8 -d out $(find src -name "*.java")
$  java -cp out com.equipo1.gestionacademica.Main
```

En Windows (PowerShell), el primer comando se escribe así:

```
>  javac -encoding UTF-8 -d out (Get-ChildItem -Recurse src -Filter *.java).FullName
```

Salida real (fragmento) de la ejecución del 8 de noviembre de 2026 con OpenJDK 25.0.3:

```
====================================
PROYECTO EQUIPO 1
GESTIÓN ACADÉMICA Y CARGA DOCENTE
====================================
…
PRUEBAS DE COLA DE SOLICITUDES
[EXITOSA] Dequeue devuelve la primera solicitud FIFO
   Detalle: Operación correcta.
[EXITOSA] Procesar solicitud ya procesada lanza excepción
   Detalle: Excepción esperada: DatoInvalidoException
…
RESUMEN GENERAL DE PRUEBAS
====================================
Pruebas exitosas: 164
Pruebas fallidas: 0
```

12 DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

La orden del examen pide declarar de forma explícita en qué partes se usó IA:

PARTE	¿IA?	DETALLE
Código fuente paquetes `modelo`, `estructuras`, `excepciones` y el sistema	SÍ	Se usó IA como apoyo para escribir el código. El equipo lo revisó, lo compiló y comprobó que funciona con las 164 pruebas de la sección 10.

A ANEXO: LAS 164 PRUEBAS

Todas en el orden en que se ejecutan. **P** positiva · **N** negativa (espera una excepción) · **B** borde. Las 164 dieron **✓ EXITOSA**.

ASIGNATURAS `pruebasAsignaturas() · 14 pruebas`

1	Insertar asignatura válida	P ✓	2	Insertar asignatura duplicada	N ✓
3	Insertar asignatura con horas en cero	N ✓	4	Insertar asignatura con horas negativas	N ✓
5	Insertar asignatura con nombre vacío	N ✓	6	Insertar asignatura nula	N ✓
7	Buscar asignatura existente	P ✓	8	Buscar asignatura inexistente	N ✓
9	Actualizar horas de asignatura existente	P ✓	10	Actualizar horas con valor negativo	N ✓
11	Actualizar horas de asignatura inexistente	N ✓	12	Eliminar asignatura existente	P ✓
13	Eliminar asignatura inexistente	N ✓	14	Verificar total de asignaturas y horas	P ✓

PROFESORES `pruebasProfesores() · 28 pruebas`

15	Contar profesores destacados en lista vacía	B ✓	16	Insertar profesor nulo	N ✓
17	Crear profesor con nombre vacío	N ✓	18	Crear profesor con categoría vacía	N ✓
19	Crear profesor con grado científico vacío	N ✓	20	Crear profesor con edad negativa	N ✓
21	Crear profesor con evaluación negativa	N ✓	22	Crear profesor con evaluación mayor que 100	N ✓
23	Agregar asignatura duplicada a un profesor	N ✓	24	Insertar profesor válido Ana	P ✓
25	Insertar profesor válido Bruno	P ✓	26	Insertar profesor válido Carla	P ✓
27	Insertar profesor válido Diego	P ✓	28	Insertar profesor duplicado Ana	N ✓
29	Buscar profesor existente	P ✓	30	Buscar profesor inexistente	N ✓
31	Actualizar evaluación válida	P ✓	32	Actualizar evaluación inválida	N ✓
33	Actualizar edad negativa	N ✓	34	Actualizar categoría vacía	N ✓
35	Verificar total de horas de Diego	P ✓	36	Verificar orden alfabético de profesores	P ✓
37	Contar profesores destacados iniciales	B ✓	38	Eliminar profesor destacado Carla	P ✓
39	Contar profesores destacados después de eliminar a Carla	P ✓	40	Eliminar profesor destacado Diego	P ✓
41	Contar profesores destacados después de eliminar a Diego	P ✓	42	Eliminar profesor inexistente	N ✓

DEPARTAMENTOS `pruebasDepartamentos() · 23 pruebas`

43	Contar profesores destacados en lista vacía de departamentos	B ✓	44	Insertar departamento nulo	N ✓
45	Crear departamento con nombre vacío	N ✓	46	Insertar departamento válido Informática	P ✓
47	Insertar departamento válido Matemática	P ✓	48	Insertar departamento duplicado Informática	N ✓
49	Buscar departamento existente	P ✓	50	Buscar departamento inexistente	N ✓
51	Agregar profesor Ana a Informática	P ✓	52	Agregar profesor Carla a Informática	P ✓
53	Agregar profesor Bruno a Matemática	P ✓	54	Agregar profesor Diego a Matemática	P ✓

55	Agregar profesor duplicado Ana en Informática	N	✓	56	Buscar profesor en departamento	P	✓
57	Buscar profesor inexistente en departamento	N	✓	58	Verificar orden de inserción de departamentos	P	✓
59	Verificar total de profesores por departamento	P	✓	60	Contar destacados por departamento Informática	P	✓
61	Contar destacados por departamento Matemática	P	✓	62	Contar profesores destacados total en la multilista	P	✓
63	Eliminar departamento Matemática	P	✓	64	Contar profesores destacados después de eliminar Matemática	P	✓
65	Eliminar departamento inexistente	N	✓				

PILA DE HISTORIAL `pruebasPila() · 24 pruebas`

66	Pila inicia vacía	B	✓	67	Pop en pila vacía	N	✓
68	Peek en pila vacía	N	✓	69	Push registro nulo	N	✓
70	Crear registro con turno inválido	N	✓	71	Crear registro con descripción vacía	N	✓
72	Crear registro con departamento vacío	N	✓	73	Crear registro con profesor vacío	N	✓
74	Crear registro con tipo de modificación vacío	N	✓	75	Crear registro con valor anterior nulo	N	✓
76	Crear registro con valor nuevo nulo	N	✓	77	Push cambio de evaluación válido	P	✓
78	Push cambio de horas de asignatura válido	P	✓	79	Verificar total de registros después de push	P	✓
80	Peek devuelve el último cambio	P	✓	81	Peek no modifica el total de la pila	P	✓
82	Pop devuelve el último cambio	P	✓	83	Verificar total después de primer pop	P	✓
84	Pop devuelve el cambio anterior	P	✓	85	Pila queda vacía después de pops	B	✓
86	Listar pila vacía	B	✓	87	Push después de vaciar la pila	B	✓
88	Listar pila con un registro	P	✓	89	Verificar orden LIFO en listado	P	✓

COLA DE SOLICITUDES `pruebasCola() · 32 pruebas`

90	Cola inicia vacía	B	✓	91	Dequeue en cola vacía	N	✓
92	Peek en cola vacía	N	✓	93	Encolar solicitud nula	N	✓
94	Crear solicitud con id inválido	N	✓	95	Crear solicitud con departamento vacío	N	✓
96	Crear solicitud con profesor vacío	N	✓	97	Crear solicitud con tipo vacío	N	✓
98	Crear solicitud con tipo inválido	N	✓	99	Crear solicitud de horas sin asignatura	N	✓
100	Crear solicitud de evaluación con asignatura	N	✓	101	Crear solicitud con edad no numérica	N	✓
102	Crear solicitud con edad negativa	N	✓	103	Crear solicitud con evaluación inválida	N	✓
104	Crear solicitud con horas negativas	N	✓	105	Encolar solicitud válida de evaluación	P	✓
106	Encolar solicitud válida de horas	P	✓	107	Verificar total después de encolar	P	✓
108	Peek devuelve la primera solicitud	P	✓	109	Peek no modifica el total de la cola	P	✓
110	Dequeue devuelve la primera solicitud FIFO	P	✓	111	Verificar total después de primer dequeue	P	✓
112	Dequeue devuelve la segunda solicitud	P	✓	113	Cola queda vacía después de dequeues	B	✓
114	Listar cola vacía	B	✓	115	Encolar después de vaciar la cola	B	✓
116	Verificar listado con una solicitud	P	✓	117	Verificar orden FIFO en listado	P	✓
118	Solicitud inicia pendiente	B	✓	119	Procesar solicitud cambia estado	P	✓
120	Rechazar solicitud cambia estado	P	✓	121	Procesar solicitud ya procesada lanza excepción	N	✓

SISTEMA INTEGRADO `pruebasSistema() · 43 pruebas`

122	Sistema inicia vacío	B	✓
123	Crear departamento válido	P	✓
124	Crear departamento duplicado	N	✓
125	Crear profesor válido	P	✓
126	Crear profesor duplicado	N	✓
127	Crear asignatura válida	P	✓
128	Crear asignatura duplicada	N	✓
129	Verificar conteo inicial de destacados	B	✓
130	Actualizar horas de asignatura	P	✓
131	Verificar destacado tras actualizar horas	P	✓
132	Verificar historial tras actualizar horas	P	✓
133	Deshacer cambio de horas	P	✓
134	Verificar horas restauradas	P	✓
135	Verificar historial vacío tras deshacer	B	✓
136	Actualizar evaluación directa	P	✓
137	Verificar evaluación actualizada	P	✓
138	Deshacer cambio de evaluación	P	✓
139	Verificar evaluación restaurada	P	✓
140	Registrar solicitud válida de evaluación	P	✓
141	Procesar solicitud de evaluación	P	✓
142	Verificar evaluación tras solicitud	P	✓
143	Registrar solicitud válida de horas	P	✓
144	Procesar solicitud de horas	P	✓
145	Verificar destacado tras solicitud de horas	P	✓
146	Deshacer solicitud de horas	P	✓
147	Verificar horas restauradas tras solicitud	P	✓
148	Deshacer solicitud de evaluación	P	✓
149	Verificar evaluación restaurada tras solicitud	P	✓
150	Procesar cola vacía	N	✓
151	Registrar solicitud con profesor inexistente	P	✓
152	Procesar solicitud con profesor inexistente la rechaza	P	✓
153	Verificar solicitudes vacías tras rechazo	B	✓
154	Verificar historial sin cambios tras rechazo	P	✓
155	Actualizar profesor con tipo de asignatura lanza excepción	N	✓
156	Actualizar horas de asignatura inexistente	N	✓
157	Crear segundo departamento	P	✓
158	Crear profesor en segundo departamento	P	✓
159	Crear asignatura destacada en segundo departamento	P	✓
160	Verificar destacados con segundo departamento	P	✓
161	Eliminar profesor destacado	P	✓
162	Verificar destacados tras eliminar profesor	P	✓
163	Eliminar departamento	P	✓
164	Verificar departamentos tras eliminar	P	✓